Chapter route

How does a file-system interface express persistent-data access through composable objects, names, and rights?

Prerequisites are block storage, processes and file descriptors, and basic access control.

- Distinguish file metadata, data, and handle state.
- Compare sequential, direct, and indexed access.
- Analyze boundaries between mounting, sharing, and protection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Files, names, and descriptors have different identities

Use ordinary local Linux/POSIX regular-file opens, excluding O_PATH and extra LSM revocation. Multiple names can refer to one live (filesystem,inode) object; one name can refer to different objects over time. After reclamation an inode number can be reused.

Layer	State	Sharing
Directory entry	Name → inode; the name lives here	A hard link adds another entry
Inode/file object	Size, owner, mode, data mapping, link count	All opens of that object
Open file description, OFD	Current offset, access mode, status flags	Descriptors shared through dup/fork
Process descriptor table	Integer fd → OFD; FD_CLOEXEC	Each process has its own table in this fork example

Open checks permission and creates an authorized OFD. Subsequent read/write still checks the descriptor's access mode. Changing DAC permissions with chmod does not automatically invalidate an already open local descriptor. Other protection mechanisms have their own rules.

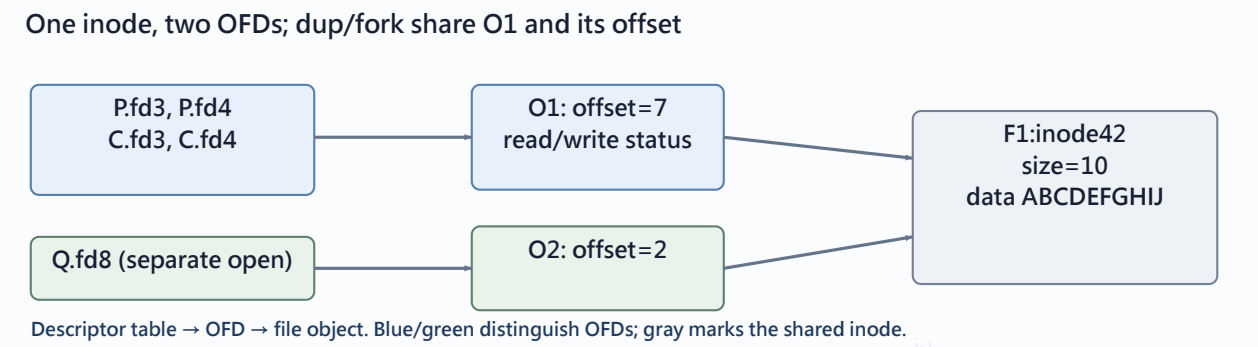
Shared offsets with data ABCDEFGHIJ

P first obtains O1 through `fd3=open(F)`, then calls `fd4=dup(fd3)`. Q independently obtains O2 through `fd8=open(F)`. P then forks C, whose fd3/fd4 also refer to O1. Operations below execute in the specified sequence; this avoids claiming historical Linux concurrent-offset behavior as a guarantee.

Step	Returned data	O1.offset	O2.offset
Initial	None	0	0
P read(fd3,3)	ABC	3	0
P read(fd4,2)	DE	5	0
Q read(fd8,2)	AB	5	2
C read(fd3,2)	FG	7	2
P pread(fd3,2, offset=1)	BC	7	2

A successful read of k bytes advances its OFD offset by k. Pread uses an explicit position without changing O1. Sharing an inode does not combine independently opened offsets.

Shared offsets with data ABCDEFGHIJ (continued)



What survives unlink, truncate, and rename?

Regular object F1:42 initially has names `/a` and `/b`, link count 2, and open description O1. Unlinking a reduces the count to 1; unlinking b reduces it to 0, but O1 can still read the object. Reclamation requires the last close and no other references. Keep name references separate from open references; real systems may also have mmap references.

`truncate(F,0)` retains the object and names but changes size to zero. Existing OFD offsets need not reset. Reading an empty file at offset 7 returns zero bytes without moving the offset back to zero.

Assume tmp and target are distinct objects in the same mount, permissions and types permit replacement, and `rename(tmp, target)` succeeds. It atomically replaces the name. New lookups of target reach the new inode; descriptors already open on the old target keep the old inode. If the names are already hard links to the same inode, Linux rename succeeds without changing them. Crossing different mounts can fail with EXDEV even when the underlying filesystem is the same. Atomic name visibility during execution is separate from surviving power loss.

Filename extensions are application conventions. They prove neither content format nor permission to execute. Linux execution additionally checks permissions and the executable/interpreter format. Renaming arbitrary bytes to `.exe` does not create a valid executable.

Logical access methods do not specify physical positions

Method	Caller specifies	Required state
Sequential	The next portion	Current position and rewind/seek rules
Direct	Relative record/block number	Fixed length or computable record boundaries
Indexed	Key	A consistent key-to-record/block index

For fixed record length L and zero-based record j, the starting byte is jL. Relative block numbers start at the beginning of the file. They are not disk LBAs; Chapter 12 handles physical mapping.

With L=32 B and blocks of 1024 B, each block holds 32 records. Record 77 starts at byte 2464, file block 2, displacement 416. A thousand records occupy 32000 B, requiring $\lceil 32000/1024 \rceil = 32$ blocks. The final block holds eight records, or 256 B, leaving 768 B of internal fragmentation. Exclude sparse files, compression, and inline data in this example.

Sparse indexes and I/O counts

The thousand records are sorted by unique integer keys 0 through 999, still at 32 records per block. The index has 32 entries of (first key, block pointer), each 16 B, totaling 512 B. For key 77, choose the greatest first key no larger than 77. Entry 64 identifies block 2; inspect keys 64 through 95 there.

If block i covers $[k_i, k_{i+1})$, sorted keys and a correct index restrict lookup to that candidate block. A gap still requires comparing the actual key. A predecessor-index hit does not prove that a record exists.

With the index in RAM, binary search takes at most $\lfloor \log_2 32 \rfloor + 1 = 6$ comparisons plus one data-block I/O. If the uncached entire index occupies one block, the bound is one index read plus one data read. This is neither a universal fixed two-I/O cost nor constant-time CPU search.

Short transfers preserve a completed prefix

Write 6000 bytes from initial file offset 100, with successful results 2500,1500,2000:

Call	Buffer start	Requested	Returned	Total done / next offset
1	0	6000	2500	2500 / 2600
2	2500	3500	1500	4000 / 4100
3	4000	2000	2000	6000 / 6100

Maintain $0 \le done \le N$: buffer[0:done] has been delivered in order, and only [done:N] remains. A successful k>0 advances done by k and reduces the remainder. If every iteration makes positive progress, at most N one-byte successes complete the request. Errors do not guarantee eventual success. Retry EINTR only under its no-data-transferred condition; wait for readiness on EAGAIN; never spin indefinitely on zero progress.

A read near EOF can return a short result; a later positive-count request returns zero. A zero-count request returning zero does not prove EOF. After a read error, the offset may not be inferable. The table assumes successful outcomes and does not invent offset advances for errors.

Append atomicity and positioned I/O

Separate `lseek(SEEK_END)` and write calls are not one atomic step. P and Q can both observe size 100 and write at offset 100, allowing the later write to overwrite the earlier one. O_APPEND combines EOF selection with that write within the interface's implementation scope. In particular, an NFS emulation need not provide the same promise as a local filesystem.

Even if each append write is atomic, a record split into A 1 and A 2 can interleave as A 1, B1, A 2. Whole-record guarantees require framing and locking or a single writer. Regular files are not pipes, so PIPE_BUF is not their atomicity theorem.

Pread/pwrite carry an explicit offset and avoid another thread changing a shared cursor between lseek and read. They do not serialize overlapping data writes. The pwrite scenario here excludes O_APPEND; Linux has a special append behavior for their combination.

Directory structures and their costs

A directory maps names to objects. Names are unique within one directory but can repeat across directories. Lookup, create, unlink, rename, and enumeration are interfaces, not a requirement to use a linear list.

Structure	Capability	Cost or limitation
Single level	One shared naming domain	Collisions and no grouping
Two level	A directory per user	Awkward sharing across users
Tree	Paths and subtree grouping	No direct shared child under a unique-parent rule
DAG	Multiple paths/names to one object	Reference counting and duplicate visits
General graph	Cycles	Termination, reclamation, cycle detection

Directory structures and their costs (continued)

A successful hard link adds an entry to the same inode within a mounted filesystem, with one copy of the data. Linux can reject a link across different mounts with EXDEV even when the underlying filesystem is the same; permissions and object type must also permit it. A symlink is a separate object storing a pathname; it can cross filesystems, dangle, or cycle. Ordinary users cannot create arbitrary directory hard links. A conceptual DAG does not authorize unrestricted Linux directory hard links.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why reference counts cannot reclaim cycles

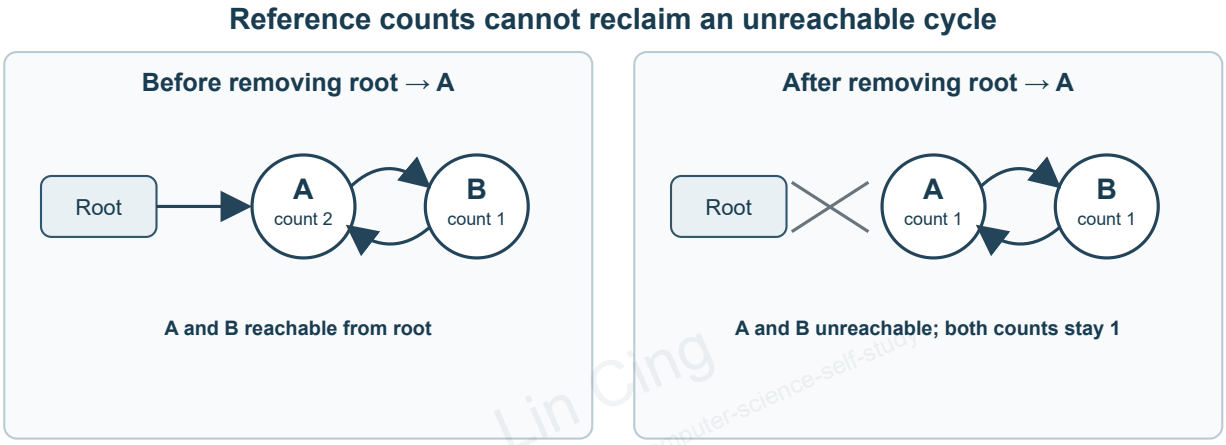
Consider only the abstract name/edge graph, excluding held opens. Let A→B, B→A, and root→A. Counts are A=2, B=1. Removing root→A leaves A=1, B=1. Both objects are unreachable, but neither count becomes zero.

For a finite DAG, an unreachable subgraph remains acyclic and has an indegree-zero vertex. It cannot have an incoming edge from a reachable vertex, or it would be reachable. Its reference count is therefore zero. Remove it and apply induction to reclaim every unreachable vertex. A cycle lacks that starting vertex.

A visited set prevents repeated traversal. DFS on a finite graph takes O(V+E), but traversal alone is not reclamation. Mark/sweep marks from roots and reclaims unmarked objects, requiring coordination with concurrent graph changes. A symlink traversal limit terminates one resolution; it does not prove all data reachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why reference counts cannot reclaim cycles (continued)



Counts include graph edges only, not open or mmap references.

Path resolution cannot start with textual cancellation

Let `/a/link` store absolute target `/x/y`; all required directories exist and are searchable. Resolve `/a/link/../b`:

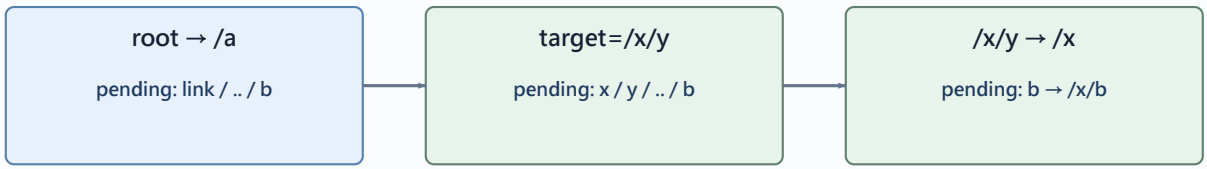
Step	Current object	Pending components	Action
0	Root	a, link,.., b	Start at process root
1	/a	link,.., b	Read link as a symlink
2	Root	x, y,.., b	Absolute target resets root; preserve suffix
3	/x/y	.., b	Resolve x and y
4	/x	b	Apply .. to resolved object
5	/x/b	Empty	Complete

Canceling `link/..` first incorrectly gives `/a/b`. Resolve a relative symlink target from the directory containing that link, not the process cwd. Each next component requires search permission on the current directory. Whether to follow a final symlink depends on the operation.

Path resolution cannot start with textual cancellation (continued)

/a/link/../b: expand the symlink before resolving ..

Target=/x/y. Blue = original path; green = expansion. Preserve the suffix.



A symlink is a separate object. An absolute target resets root; append the original suffix.

Wrong: cancel link/.. to get /a/b. Correct: /x/y/.. reaches /x, then /x/b.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TOCTOU can change the object behind a name

P checks that `/tmp/out` is a safe ordinary file. An attacker replaces it with a symlink to a sensitive file before P opens the same string. Two resolutions can reach different objects because the name graph changed; symlinks themselves are not universally invalid.

A trusted directory descriptor with openat fixes the relative starting point. O_NOFOLLOW constrains only the final symlink and does not authenticate intermediate components. Later Linux openat2 resolution restrictions can constrain the whole walk; do not assign that later interface to Linux 3.2.

Complete the operation through the same protected reference or appropriately restricted resolution. After open, fstat examines the opened object. Still define acceptable owner/type and allowed open side effects. Checking safety after O_TRUNC is too late to prevent truncation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

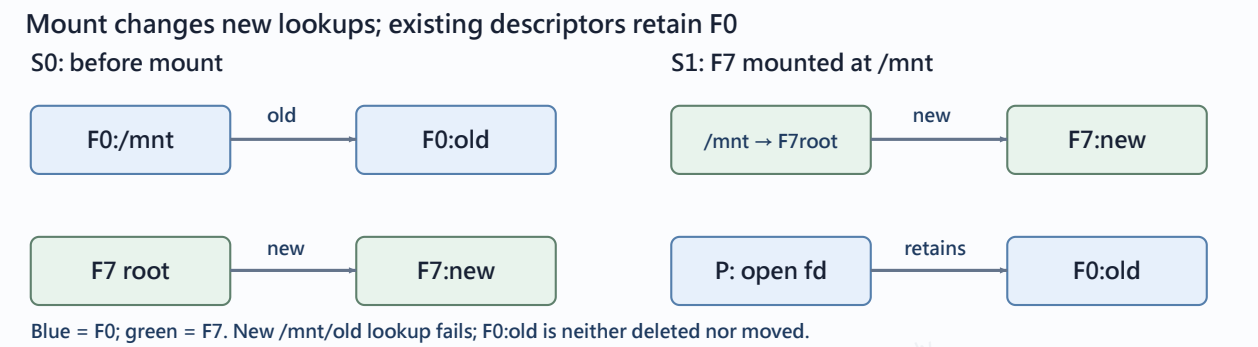
Mount changes a resolution edge

F0's `/mnt` initially contains old but not new; F7's root contains new but not old. All path ancestors and relevant directories are searchable, without other permission restrictions that prevent resolution. After mounting F7 at /mnt, a new lookup of `/mnt/new` reaches F7:new, while `/mnt/old` returns ENOENT. F0's old remains allocated and becomes visible after unmount.

Observation	Before mount	After mount
New lookup /mnt/old	F0:old	ENOENT in F7
New lookup /mnt/new	ENOENT	F7:new
Previously opened F0:old descriptor	F0:old	Still F0:old

The namespace selects a mount map. The same string can reach different objects in different namespaces without copying their data. Blue denotes F0, green F7, and gray the old content hidden from new lookups.

Mount changes a resolution edge (continued)



Mount crossings, .., and busy references

In this Linux-style model, crossing a mount point enters the mounted root. Applying `..` there returns to the mount point's parent, rather than arbitrarily remaining at the underlying filesystem root. At the process root itself, `..` stays at root.

Let `/a/link` target `../b` and mount F7 at `/b`. Resolving `/a/link` follows root→a→link target→a/..=root→b→F7 root, then stops. Without an input filename, it cannot invent `/file` as a suffix.

Normal unmount can report busy for active cwd, open-file, or submount references under the system's rules. Linux lazy detach first removes the namespace attachment, then reclaims after references disappear. It neither immediately invalidates every descriptor nor guarantees pending I/O success.

A mount point is not a symlink. The namespace mount map controls mounting; a symlink is a directory object storing a target string. Both can occur in one path walk without becoming the same entry type.

Identity, ordering, visibility, and durability

The same user name on two hosts need not map to the same UID. Inconsistent AUTH_SYS-style numerical mappings can authorize the wrong principal. A network address or declared UID does not replace trusted authentication. NIS, LDAP, and Active Directory are naming/identity infrastructure examples. LDAP is a directory protocol; its use alone proves neither encryption nor correct authentication.

A file protocol must separately define who may operate, operation order, when another client rereads new data, and what survives a crash. NFSv3 basic file operations do not need OPEN state to identify each READ/WRITE, but still have caches and stable-write semantics. NFSv4 adds stateful mechanisms. Stateless does not mean no state of any kind or automatic security.

Equal paths can reach different mounted objects. Clients of one object can temporarily observe different cached/session versions. Successful write, successful lock acquisition, another client's visibility, and media durability are four different events.

One history under immediate and session visibility

Start at value 0. A and B open, A writes 1, B reads before A closes, A closes, B reads within its existing session, then newly opened C reads. Execute in order and ignore errors/data races. The immediate column is a single coherent image abstraction, not a claim about every network cache.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One history under immediate and session visibility (continued)

One history: immediate visibility and session snapshots

Event	One shared image	Session snapshot
A, B open	A, B see 0	A and B each hold 0
A writes 1	image becomes 1	A private 1; B still 0
B reads	returns 1	returns 0
A closes / publishes	image remains 1	published image becomes 1
B reads, same session	returns 1	old snapshot returns 0
C opens / reads	returns 1	new snapshot returns 1

Ordered example only; multi-writer conflicts need a separate rule. Not a claim about all network filesystems.

Session snapshots do not change an already open instance when another closes. Multiple-writer conflict and last-close rules need separate definitions. Immutable sharing prohibits changing contents or reusing the unique name, simplifying caching but expressing updates as new objects/names.

Advisory locks provide conditional exclusion

A holds an exclusive lock on [0,100). Cooperating B waits for a shared or exclusive lock on the same range; a nonoverlapping lock on [100,200) can proceed. Half-open ranges overlap exactly when $\max(start) < \min(end)$, so these two ranges do not overlap.

Time	A	Cooperating B	C without a lock
0	Acquires [0,100) X	None	None
1	Writes version 8	Waits for overlapping lock	Can directly write the same data
2	fsync succeeds	Still waits	May violate A/B expectations
3	Unlocks	Acquires, then validates/reads under cache protocol	None

Assume every conflicting operation first obtains a compatible lock. If two writers executed inside the same range simultaneously, both would hold incompatible X locks, contradicting the lock service. C's uncooperative access breaks that premise. Locks alone establish neither cache invalidation nor durability; the shared-filesystem contract must provide those rules.

Atomic publication is separate from durable publication

Assume a local Linux filesystem supporting same-mount atomic replacement and directory fsync, and a device honoring flush. Tmp and target are distinct objects in one directory; permissions, types and other conditions allow replacement. Preserve the old target while preparing the new version:

1. Create a unique tmp, write all data with short-write handling, and set required metadata.
2. Successfully fsync(tmpfd) to persist contents and necessary metadata.
3. Successfully rename(tmp, target). New lookups reach the new inode; existing old descriptors retain the old version.
4. Successfully fsync the parent-directory descriptor before promising that the new name survives a crash. Handle every error.

Step 3 avoids a runtime interval in which target is absent. Step 4 concerns the durable name. Before it finishes, this chapter does not claim that every filesystem must recover to a specified old or new name. Two fsync calls are not a cross-filesystem transaction; rename can fail with EXDEV.

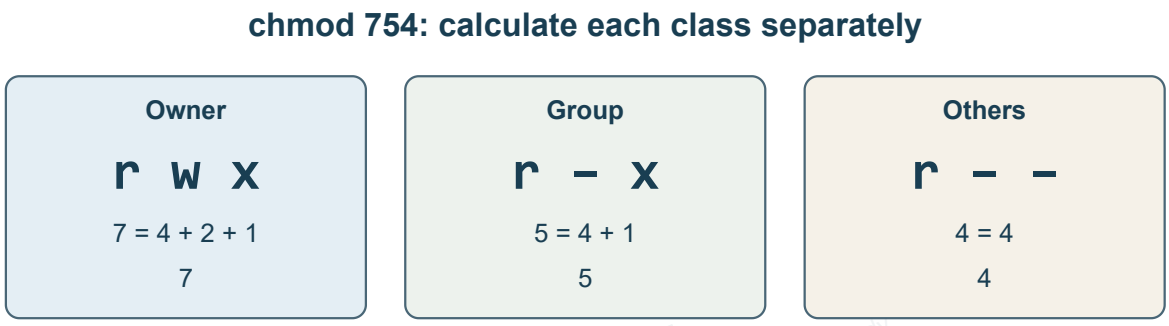
File and directory rwx govern different operations

First exclude root/capability privilege, ACLs, sticky rules, and other controls. Linux compares filesystem UID (fsuid), filesystem GID (fsgid), and supplementary groups; assume here fsuid/fsgid equal the effective UID/GID. Select one mode class: owner if matched, otherwise a matching filesystem or supplementary group, otherwise other. Do not combine all three classes. With r=4, w=2, x=1, every requested bit must be present in the selected class.

Right	Regular file	Directory
r	Read data	Enumerate names
w	Change data	Add, remove, rename entries, normally also requiring x
x	Request execution with valid format and other conditions	Search/traverse a known name

With D mode0711 and F mode0644, a nonowner can traverse D and read known F but cannot list D. With D mode0733, other has wx and can add/remove known names without listing. Unlink primarily checks parent-directory wx, not the file's w bit. Sticky directories add identity restrictions.

chmod 754: three groups of permission bits



`chmod 754 target` sets the basic mode bits to owner `rwx`, group `r-x`, and others `r--`. Within each group `r=4`, `w=2`, and `x=1`: group `5=4+1`, so it has read and execute but not write.

Access follows the applicable owner, group, or others class; the three columns are not simply added together. For the examples below, assume an ordinary unprivileged user and a local Linux file, with no additional ACL, LSM restrictions, or mount restrictions. ACLs, special permissions, and privileges can change the result.

The meaning of rwx depends on the object

Object	r	w	x
Regular file	Read contents	Write contents	Request execution, subject to format, interpreter, and other checks
Directory	List entry names	Modify entries, usually also requiring x	Search the directory and resolve names inside it

Let `/work` be a 754 directory and assume all ancestors are searchable. A user in the applicable group class has `r-x`: they may list and search, but cannot create an entry using those permissions alone. An others-class user has `r--`: they can read entry names but cannot resolve `/work/a` without directory search permission. The owner has `rwx`; creation or deletion remains subject to other conditions such as the sticky bit and mount restrictions.

Suppose a process successfully opened a separate regular file for writing before its mode was changed. On a local Linux regular file, removing that process's write mode bit later does not automatically close its existing descriptor; a new open is evaluated against the permissions then in force. This example excludes additional LSM revocation, network protocol and device-specific behavior. The existing descriptor's access mode still limits later operations.

Linux POSIX-style ACL mask and selection order

Use Linux access ACLs, not Windows deny ACLs, excluding privilege/LSM. Here filesystem UID/GID (fsuid/fsgid) equal effective UID/GID, and group matching also includes supplementary groups. Owner alice and file group dev have entries user::rw-, user:bob:-w-, group::r--, group:qa:-w-, mask::rw-, other::r--.

1. If fsuid matches owner, check user:: directly without the mask.
2. Otherwise, a matching named-user entry must contain the entire requested set, as must the mask. Failure denies without falling back to groups/other.
3. Otherwise consider matching owning/named groups. At least one matching entry must contain the whole requested set, and the mask must contain it. A matched but insufficient group class denies without falling back to other.
4. Only if no preceding identity matches, use other without the mask.

Bob has only w even if he belongs to dev; group r does not supplement his named entry. Changing mask to r leaves him no rights, without falling back to other r. Alice keeps rw. With a mask entry, displayed mode group-class bits represent the mask, not necessarily group::.

Multiple groups do not authorize every combined request

Carol belongs to dev and qa. Here dev=r, qa=w, mask=rw. A read request passes through dev; a write passes through qa. A combined rw request, such as O_RDWR, fails because no single matching group entry contains both rights.

Subject/request	Selected path	Decision
Alice/rw	Owner rw	Allow without mask
Bob/r	Named user w	Deny without group fallback
Bob/w	Named user w and mask rw	Allow
Carol/r	dev r and mask rw	Allow
Carol/w	qa w and mask rw	Allow
Carol/rw	Each matching group lacks a right	Deny
No matching identity/r	Other r	Allow

Changing mask to r also denies Carol's write. Apply the algorithm to the complete current request rather than OR-ing every displayed entry. Named-user precedence is not a general deny-entry composition rule.

Default ACLs, umask, and existing handles

Without a default ACL, creating with mode0666 and umask0027 gives $0666 \,\&\, \sim 0027 = 0640$. Umask removes rights and does not add unrequested execute permission.

A directory default ACL is inherited as the new object's access ACL and restricted by the supplied create mode for owner/group-class/other. Do not additionally apply umask unconditionally. A default ACL initializes new objects without retroactively changing old files or replacing the ordinary access-check algorithm.

Opening a writable descriptor before chmod0000 generally retains write authority for an ordinary local Linux regular file, while a new open is denied by DAC. Immediate revocation requires a separately defined mechanism or closing/revoking references. Changing displayed permissions does not recall every authorized reference. See acl(5).

Four traces for the same file

Trace	Record	Typical mistake
Names	Root/cwd, components, symlink suffix, mount map	Textually cancel link/..
Objects	Filesystem/inode, links, open references	Free after unlinking the last name
Access	OFD offset/access mode, actual transfer results	Assume dup offsets are independent
Authority	Directory search, selected ACL class, mask, authorization time	Treat a displayed w as unconditional write access

Atomic publication adds a persistence trace: complete tmp data→file fsync→rename→directory fsync. The name graph and durable storage have different publication points; one success label cannot represent all of them.

Choose operations from their counterexamples

Requirement	Condition that matters
Read at a position	pread plus short-read/error handling
Append a whole record	framing and whole-record lock or one writer
Remove or confine a name	parent permissions; constrain the entire path
Coordinate client updates	cooperating writers and a visibility protocol
Preserve new name after crash	file and directory persistence under the stated flush contract

Chapter 12 implements these name, object, and handle contracts.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 11. Linux open(2); rename(2); acl(5); path_resolution(7); credentials(7).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study